RTOS IRTX
开发指南

版本号: 1.5

发布日期: 2025.7.25

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2022.10.28	XAA0248	初始版本
1.1	2023.3.28	XAA0248	添加 XR875 的说明
1.2	2023.12.19	XAA0311	添加 MR527 的说明
1.3	2024.8.22	AWA2215	添加 MR536 的说明
1.4	2024.8.30	AWA2215	添加 T536 的说明
1.5	2025.7.25	AWA2231	添加 F135 的说明

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 2
1.4.1 标志说明 2
1.5 相关术语介绍 2

2 模块介绍 **3**
2.1 模块功能介绍 3
2.2 模块配置介绍 4
2.3 模块源码结构 4

3 模块接口说明 **5**
3.1 内部调制原理 5
3.2 载波频率设置 5
3.3 数据发送流程 6
3.4 设置载波占空比 6
3.5 设置载波频率 6
3.6 接口列表 7
3.7 接口使用说明 7
3.7.1 IRTX 初始化接口 7
3.7.2 IRTX 设置载波占空比接口 8
3.7.3 IRTX 设置载波频率接口 8
3.7.4 IRTX 使能发送接口 8

4 模块使用范例 **9**

5 FAQ **12**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　红外发射电路 3
图 2-2　红外接收电路 3
图 2-3　IRTX menuconfig 4
图 3-1　内部调制原理方框图 5
图 3-2　IR-TX 驱动数据发送流程 6
图 3-3　IR-TX 驱动设置载波频率流程 7

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

介绍 RTOS 中 IRTX 驱动的接口及使用方法，为 IRTX 使用者提供参考。

1.2 目标读者

IRTX 驱动的开发/维护人员。

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
R128	FreeRTOS	hal_cir_tx.c
XR875	FreeRTOS	hal_cir_tx.c
MR527	FreeRTOS	hal_cir_tx.c
MR536	FreeRTOS	hal_cir_tx.c
T536	FreeRTOS	hal_cir_tx.c
F135	FreeRTOS	hal_cir_tx.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4 文档约定

1.4.1 标志说明

注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.5 相关术语介绍

术语	解释说明
Sunxi	指 Allwinner 的一系列 SOC 硬件平台
IR	Infrared Remote，红外模块
TX	发送
占空比	决定了一个周期内 IRTX 信号高低的比例，即一个周期内的平均电压
极性	决定了高电平有效或者低电平有效
开关	控制 IRTX 信号是否输出

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能介绍

红外是一种电磁波，可以实现数据的无线传输，由发送和接收两个部分组成。发送端对红外信号进行脉冲编码，接收端完成对红外信号的脉冲解码。红外遥控协议有多种，如NEC、SIRC、RC-5等，这些协议都比较简单，基本都是以脉冲宽度或脉冲间隔来编码。当遥控器按下按键时，遥控器逻辑单元会产生一个完整的脉冲波形，包含遥控指令的信息，即红外传输的基带信号。这个波形被送到遥控器的调制单元，经调制单元调制成高频红外电磁波信号，由发光二极管发射出去，如下图所示。

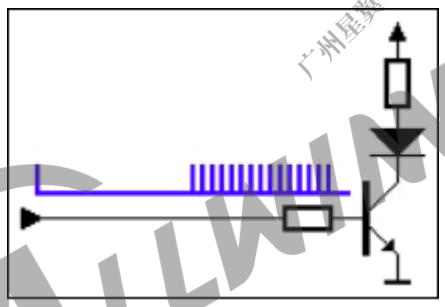

图 2-1: 红外发射电路

红外电磁波信号一般使用一体化接收头接收，同时完成信号的解调和放大，其输出信号就是红外的基带脉冲信号。解调后的信号可直接送入信号处理单元，由处理单元对脉冲波形进行解码，典型红外接收电路如下图所示。

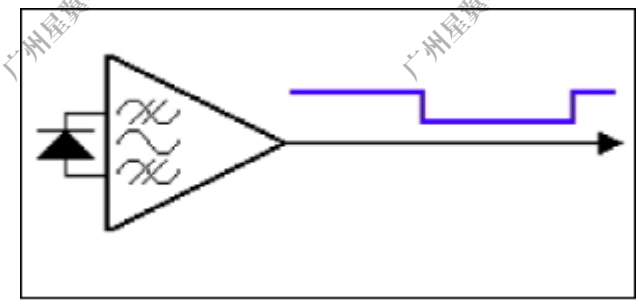

图 2-2: 红外接收电路

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2 模块配置介绍

```
                                    CIR_TX Devices
 submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.  Pr
ch.  Legend: [*] built-in  [ ] excluded  <M> module  < > module capable


            [*] enable CIR_TX driver
                CIR_TX option features  --->
```

图 2-3: IRTX menuconfig

2.3 模块源码结构

IRTX 模块源码结构如下所示：

```
rtos-hal/
|--hal/source/cir_tx/hal_cir_tx //hal层接口代码
|--include/hal/sunxi_hal_cir_tx.h //头文件
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块接口说明

需要包含头文件:

```
#include <sunxi_hal_cir_tx.h>
```

3.1 内部调制原理

内部调制原理如下图所示。

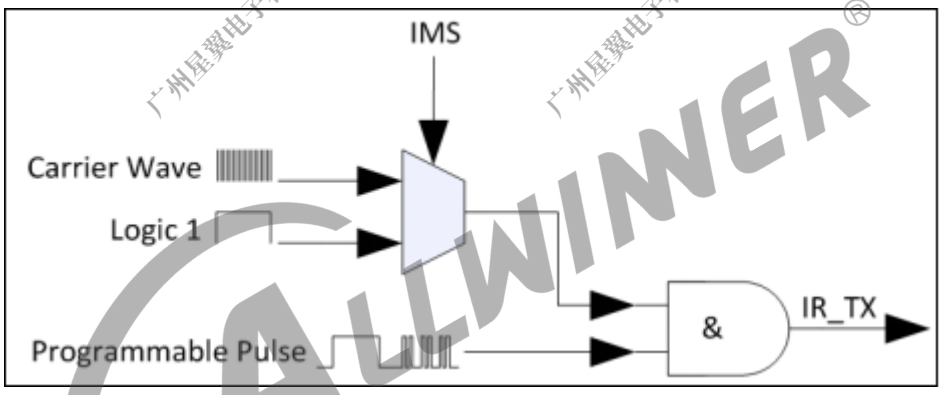

图 3-1: 内部调制原理方框图

IMS：Internal Modulation Select，选择使用内部调制或不调制。软件需设置使用 IMS。

3.2 载波频率设置

载波频率计算公式为：Fc=Fclk/[(RFMC+1)*(DRMC+2)] （公式-1）

其中

```
Fc为载波频率；
Fclk为Sunxi IR-TX时钟源，目前配置为12MHz；
RFMC为载波分频系数，由MCR寄存器（0x04）bit[7:0]设定；
DRMC为载波占空比设置，由GLR寄存器（0x00）bit[6:5]设定，可配置1/2、1/3或1/4。
```

通常，载波占空比 DRMC 和载波频率 Fc 由应用层设定，因此设置载波占空比即转换为设置 GLR bit[6:5]，设置载波频率即转换成设置 MCR bit[7:0]，由公式 1 变换得计算 RFMC 的公式为：RFMC=Fclk/[Fc*（DRMC+2）]-1 （公式-2）

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3 数据发送流程

IR-TX 驱动数据发送流程如下图所示：

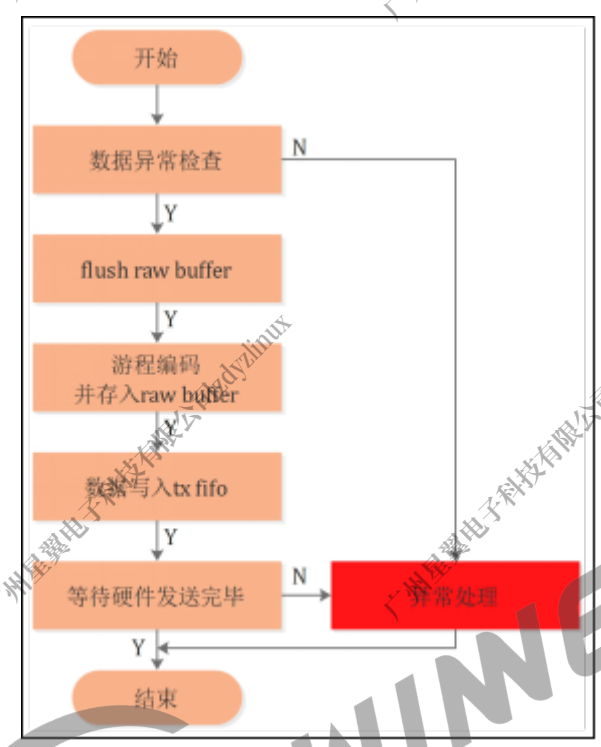

图 3-2: IR-TX 驱动数据发送流程

3.4 设置载波占空比

由于硬件只支持 1/4、1/3 和 1/2 三挡占空比设置，因此 Sunxi IR-TX 驱动设置载波占空比的流程比较简单：上层传递一个 0~100 的数值，如果该数值小于 30，则设定占空比为 1/4；如果该数值大于 30 且小于 40，则设定占空比为 1/3；如果该数值大于 40，则设定占空比为 1/2。最后更新硬件配置。

3.5 设置载波频率

IR-TX 驱动设置载波频率流程如下图所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

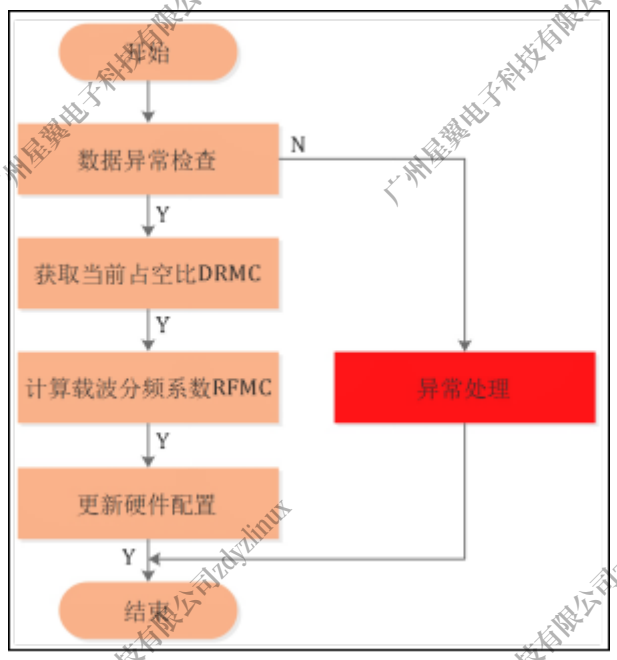

图 3-3: IR-TX 驱动设置载波频率流程

3.6 接口列表

IRTX HAL 层提供的接口列表如下：

```
cir_tx_status_t hal_cir_tx_init(struct sunxi_cir_tx_t *cir_tx);
void hal_cir_tx_set_duty_cycle(int duty_cycle);
void hal_cir_tx_set_carrier(int carrier_freq);
void hal_cir_tx_xmit(unsigned int *txbuf, unsigned int count);
```

3.7 接口使用说明

3.7.1 IRTX 初始化接口

- 原型：cir_tx_status_t hal_cir_tx_init(struct sunxi_cir_tx_t *cir_tx);
- 功能：IRTX 模块初始化，主要完成 clk 初始化
- 参数：无
- 返回值：
  - 0 代表成功
  - -1 代表失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.7.2 IRTX 设置载波占空比接口

- 原型：void hal_cir_tx_set_duty_cycle(int duty_cycle);
- 功能：配置 IRTX 模块占空比
- 参数：占空比大小
- 返回值：无

3.7.3 IRTX 设置载波频率接口

- 原型：hal_cir_tx_set_carrier(int carrier_freq);
- 功能：设置载波频率
- 参数：载波频率大小
- 返回值：无

3.7.4 IRTX 使能发送接口

- 原型：void hal_cir_tx_xmit(unsigned int *txbuf, unsigned int count);
- 功能：发送 IRTX 数据
- 参数：
  - txbuf 代表数据 buf
  - count 代表数据长度
- 返回值：无

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 模块使用范例

可参考驱动 APIs 测试代码（hal/test/cir_tx/hal_cir_tx.c），具体实现如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <hal_cmd.h>
#include "sunxi_hal_cir_tx.h"

#define NS_TO_US(nsec)     ((nsec) /  1000)
#define NEC_NBITS       32
#define NEC_UNIT        562500 /* ns. Logic data bit  pulse length */
#define NEC_HEADER_PULSE   (16 * NEC_UNIT) /* 9ms. 16 * Logic data bit pulse length*/
#define NEC_HEADER_SPACE   (8 * NEC_UNIT)  /* 4.5ms */
#define NEC_BIT_PULSE      (1 *  NEC_UNIT)
#define NEC_BIT_0_SPACE    (1 * NEC_UNIT)
#define NEC_BIT_1_SPACE    (3 * NEC_UNIT)
#define NEC_TRAILER_PULSE   (1 * NEC_UNIT)
#define NEC_TRAILER_SPACE   (10 * NEC_UNIT) /* even longer */
#define GPIO_IR_RAW_BUF_SIZE    128
#define DEFAULT_DUTY_CYCLE  33
#define DEFAULT_CARRIER_FREQ    38000
#define LIRC_MODE2_PULSE    0x01000000
#define LIRC_MODE2_SPACE    0x00000000
#define LIRC_VALUE_MASK     0x00FFFFFF
#define LIRC_MODE2_MASK     0xFF000000

#define LIRC_PULSE(val) (((val)&LIRC_VALUE_MASK) | LIRC_MODE2_PULSE)
#define LIRC_SPACE(val) (((val)&LIRC_VALUE_MASK) | LIRC_MODE2_SPACE)

uint32_t tx_raw_buf[GPIO_IR_RAW_BUF_SIZE];

static  int  nec_modulation_byte(uint32_t *buf, uint8_t code)
{
    int  i  = 0;
    uint8_t mask = 0x01;

    while (mask) {
        if   (code & mask) {
            /*  bit  1 */
            *(buf + i )  = LIRC_PULSE(NS_TO_US(NEC_BIT_PULSE));
            *(buf + i  + 1)  = LIRC_SPACE(NS_TO_US(NEC_BIT_1_SPACE));
        }  else {
            /*  bit  0 */
            *(buf + i )  = LIRC_PULSE(NS_TO_US(NEC_BIT_PULSE));
            *(buf + i  + 1)  = LIRC_SPACE(NS_TO_US(NEC_BIT_0_SPACE));
        }
        mask <<= 1;
        i  += 2;
    }
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    return i;
}

static int ir_lirc_transmit_ir(uint32_t *raw_buf, size_t n)
{
    int ret, count;

    count = n / sizeof(unsigned int);
    if (count > 1024 || count % 2 == 0) {
        return -1;
    }
}

static int nec_ir_encode(uint32_t *raw_buf, uint32_t key_code)
{
    uint8_t address, reverse_address, command, reverse_command;
    uint32_t *head_p, *data_p, *stop_p;

    address = (key_code >> 24) & 0xff;
    reverse_address = (key_code >> 16) & 0xff;
    command = (key_code >> 8) & 0xff;
    reverse_command = (key_code >> 0) & 0xff;

    /* head bit */
    head_p = raw_buf;
    *(head_p) = LIRC_PULSE(NS_TO_US(NEC_HEADER_PULSE));
    *(head_p + 1) = LIRC_PULSE(NS_TO_US(NEC_HEADER_SPACE));

    /* data bit */
    data_p = raw_buf + 2;
    nec_modulation_byte(data_p, address);

    data_p += 16;
    nec_modulation_byte(data_p, reverse_address);

    data_p += 16;
    nec_modulation_byte(data_p, command);

    data_p += 16;
    nec_modulation_byte(data_p, reverse_command);

    /* stop bit */
    stop_p = data_p + 16;
    *(stop_p) = LIRC_PULSE(NS_TO_US(NEC_TRAILER_PULSE));
    *(stop_p + 1) = LIRC_PULSE(NS_TO_US(NEC_TRAILER_SPACE));

    return ((NEC_NBITS + 2) * 2 - 1);
}

int cmd_test_cir_tx(int argc, char **argv)
{
    int key_code = 0x04fb13ec;
    int i, size;
    int count = 67;
    struct sunxi_cir_tx_t *cir_tx;

    hal_cir_tx_init(cir_tx);

    hal_cir_tx_set_duty_cycle(DEFAULT_DUTY_CYCLE);
    hal_cir_tx_set_carrier(DEFAULT_CARRIER_FREQ);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    size = nec_ir_encode(tx_raw_buf, key_code);
    for (i = 0; i < size; i++) {
        printf("%d ", *(tx_raw_buf + i) & 0x00FFFFFF);
        if ((i + 1) % 8 == 0) {
            printf("\n");
        }
    }
    printf("\n");

    for (i = 0; i < size; i++)
        tx_raw_buf[i] = (tx_raw_buf[i] & 0x00FFFFFF);

    hal_cir_tx_xmit(tx_raw_buf, count);
    printf("end test!\n");

    return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 FAQ

无

广州星翼电子科技有限公司zdyzlinux

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。